MICROSERVICES IN AWS

AWS microservices enable building scalable, resilient, and independently deployable applications using small, loosely coupled services managed by autonomous teams. Key AWS services for this architecture include Amazon ECS and EKS for containers, AWS Fargate for serverless hosting, AWS Lambda for function-as-a-service, and Amazon API Gateway for communication.

Key Components & Services for AWS Microservices:

- **Compute (Hosting):**
  - **AWS Fargate:** Serverless container hosting that removes the need to manage servers.
  - **Amazon ECS & EKS:** Container orchestration for Docker containers and Kubernetes, respectively.
  - **AWS Lambda:** Serverless compute for running code without provisioning servers.
  - **AWS App Runner:** Fully managed service for quick deployment of containerized web apps.
- **Networking & Communication:**
  - **Amazon API Gateway:** Creates, secures, and manages API endpoints for services.
  - **AWS App Mesh:** Provides application-level networking for service communication.
- **Data Storage:**
  - **Amazon DynamoDB:** NoSQL database for fast, flexible, and scalable data storage.
  - **Amazon RDS & Aurora:** Managed relational database services.
- **Messaging & Integration:**
  - **Amazon SQS:** Message queuing for asynchronous communication between services.
  - **Amazon SNS:** Pub/sub messaging for event-driven architectures.

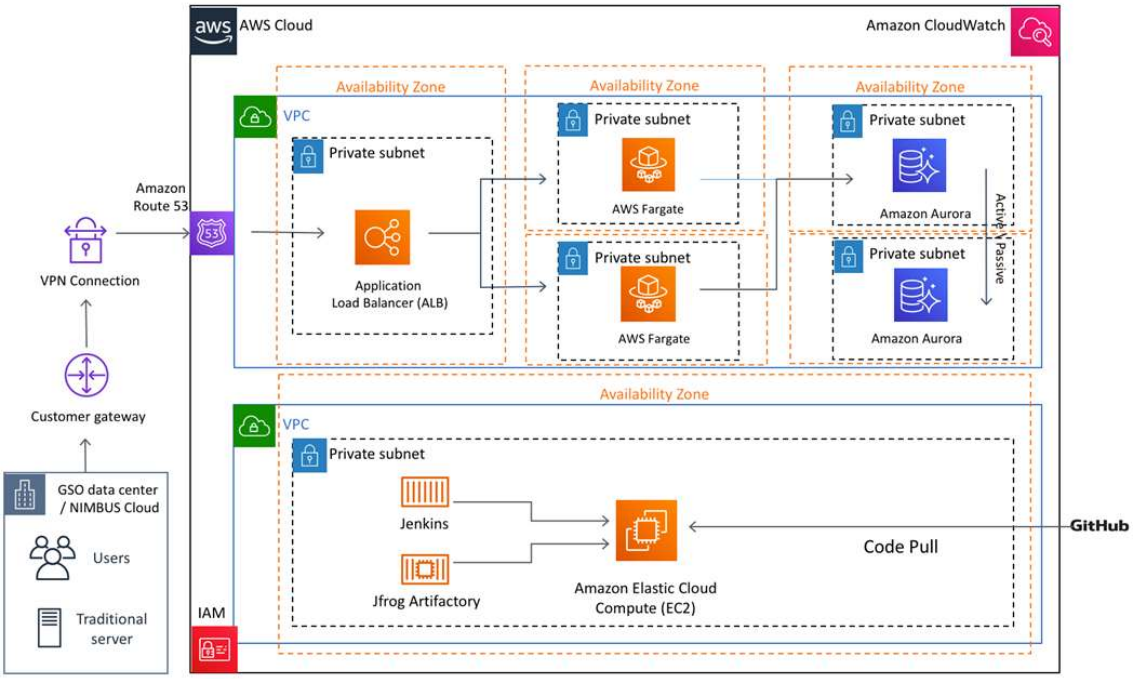
- **Monitoring & DevOps:**
    - **Amazon CloudWatch:** Monitors resource performance and logs.
    - **AWS X-Ray:** Provides distributed tracing to analyze request latency.
    - **AWS CodePipeline & CodeBuild:** Automates CI/CD workflows.

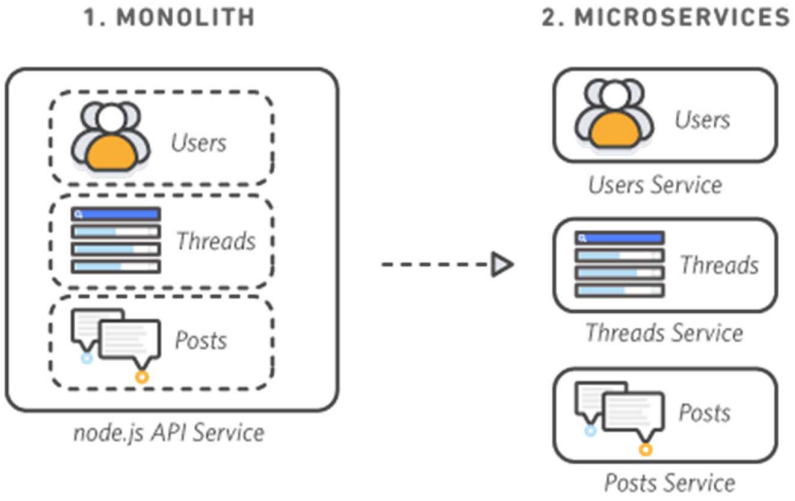
Advantages of AWS Microservices:

- **Increased Agility:** Independent development and deployment cycles.
- **Resilience:** Fault isolation ensures one service failure doesn't crash the system.
- **Scalability:** Individual services scale based on specific demand.
- **Flexibility:** Teams can choose the best tech stack for each service.

Key Architectural Patterns:

- **API-Driven:** Synchronous communication via REST APIs.
- **Event-Driven:** Asynchronous communication utilizing SQS/SNS.
- **Database per Service:** Each microservice maintains its own data store to ensure autonomy.




1. MONOLITH
Users
Threads
Posts
node.js API Service
2. MICROSERVICES
Users
Users Service
Threads
Threads Service
Posts
Posts Service